Jorge Eduardo Morales Villalobos **A01285645**
Scanners and Parsers **04/10/2026**

Flex y Bison

- Se usa en plataformas tipo Unix y se escribe en C
- Primero se genera el archivo_bison.y que se usa para generar los tokens que flex usará, luego el archivo_flex.l que genera lex.yy.c, con el output de ambos procesos se usa gcc y se produce el ejecutable.
- Ambas son gratis
- Flex, que usa Expresiones regulares => Autómata Finito No determinista (AFN) => Automata Finito Determinista(AFD), con tablas comprimidas. Y Bison que es un analizador ascendente LR con tablas LALR.
- Línea de comandos
- No sería complicado de inyectar, solo sería importante que el código sea C, pero requiere que sea puesto de una manera específica. En la primera sección todos los #include, en la segunda el codigo en C que se ejecute al reconocer un patrón con Flex, y la sección final donde se incluya el main y el yyerror.

Pest

- Cualquier plataforma donde corra Rust, y se escribe en Rust.
- Se usa un archivo .pest donde se declara la gramática. Luego en un archivo Rust se llama la biblioteca y se ejecuta. Este tipo no tiene un analizador léxico separado, los tokens se generan con la gramática.
- Gratis
- Herramientas teóricas en las que se basan: expresiones regulares, métodos sintácticos empleados, etc.
- No tiene interfaz, es una biblioteca de cargo. Pero tienen un editor en línea donde puedes probar escribir una gramática y ejecutarla.
- Difícil, no se inyecta código en la gramática solo se le declara que encontrar en términos de gramática, después declaramos en código rust caso por caso que pasa al encontrarse con cada una de las reglas declaradas.

LALRPOP

- Cualquier plataforma que corra Rust, y se escribe en Rust
- Se crea un archivo .lalrop que viene con las expresiones encontradas y sus opciones (alternativas). Se crea el código rust donde se llama process_src() y al hacer cargo build se genera el parser.
- Gratis
- LALROP, analizador ascendente LR mediante tablas LALR. Logos y Lexer(integrado de LALROP) que sirven para pasar de expresiones regulares a autómatas.
- No tiene interfaz gráfica, son bibliotecas de cargo
- Muy fácil de inyectar código, y el compilador se encarga de verificar los tipos del código insertado para manejo más claro de errores.

Referencias:

- https://crates.io/crates/pest
- https://lalrpop.github.io/lalrpop/tutorial/index.html
- https://www.gnu.org/software/bison/
- https://github.com/westes/flex

IA:

- Cómo funciona Flex Y Bison a la hora de compilar código, dame los datos específicos sobre cómo funciona y cómo se implementa en código.
- Cómo funciona Pest a la hora de compilar código, dame los datos específicos sobre cómo funciona y cómo se implementa en código.
- Cómo funciona Lalrpop a la hora de compilar código, dame los datos específicos sobre cómo funciona y cómo se implementa en código.
- Dame una guia de como implementar mi propio test para correr Lalrpop en mi maquina con una calculadora sencilla.